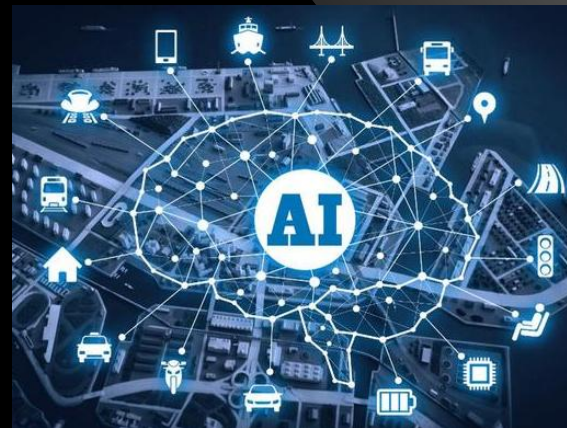


다양한 인공지능 모델 원리와 구현



이 슬라이드에서 사용한 서체 :

- Open Sans(<https://ko.cooltext.com/Download-Font-Open+Sans>)
- KoPubWorld돋움체(<http://www.kopus.org/biz/electronic/font.aspx>)



실습 환경 (선택 1)

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

1. <https://repo.anaconda.com/archive/>에서 아나콘다 다운로드 및 설치
 - 강의장 운영체제는 Windows 10, GPU 2080
 - GPU를 사용하려면 [Anaconda3-2023.03-1-Windows-x86_64.exe](#) 파일을 내려받아 설치하세요.
 - 파이썬 개발 인터프리터, 주피터노트북 개발환경, 머신러닝 및 딥러닝을 위한 패키지 등이 설치됨
2. 아나콘다 설치 후 Anaconda prompt를 **관리자 권한으로 실행**하고 패키지 설치
 - 윈도우 10에서 GPU 사용: `pip install tensorflow==2.10`
 - 윈도우 11: `pip install tensorflow` 또는 `pip install tensorflow-gpu`

실습 환경 (선택 2)

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

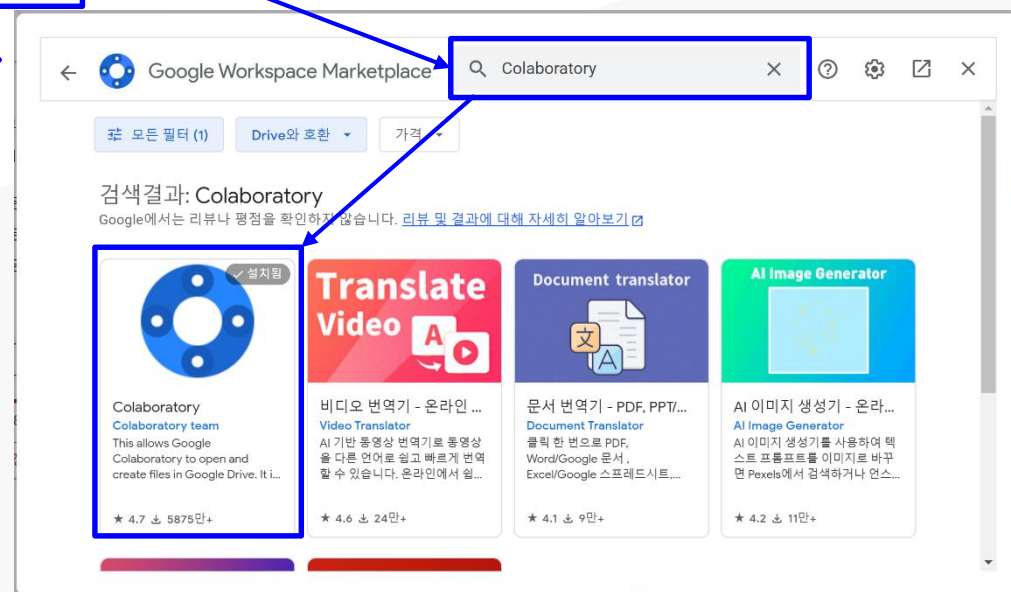
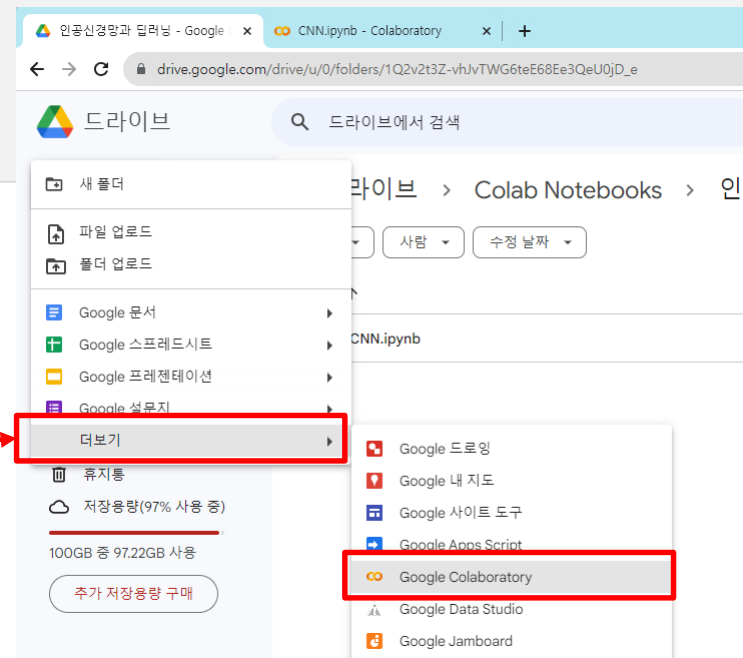
1. 개발 및 실행 환경은 Google Colab notebook을 사용합니다.

1. Chrome 브라우저에서 구글 계정으로 로그인
2. 구글 드라이브로 이동(<https://drive.google.com/>)

3. + 신규 -> 더보기 -> Google Colaboratory 선택
 - Colaboratory가 없을 경우 '연결할 앱 더보기'를 선택하고 Colaboratory를 검색하여 설치하세요.

4. 코드 작성 후 실행은 '컨트롤 + 엔터'

```
# 코랩에서 한글 그래프 적용하려면(세션 다시 시작)
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
```



코드 작성 및 실행

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

The screenshot shows a Google Colab notebook titled "누구나 할 수 있는 인공지능 모델 구현.ipynb". The interface includes a file explorer on the left, a toolbar with buttons for adding code or text, and a main area for code execution. A red box highlights the first code cell, and a blue box highlights the second code cell. A red box also highlights the "+ 코드" button in the toolbar. A blue arrow points from the text box on the right to the "+ 코드" button.

1 # #은 주석입니다.
2 # 코드를 작성하는 이 곳을 셀(Cell)이라고 부릅니다.
3 # 코드를 실행하려면 셀의 왼쪽에 있는 셀 실행 버튼을 클릭하거나 Shift+Enter 키를 누르세요.
4 # Shift+Enter로 실행하면 아래에 셀이 없으면 새로운 셀을 추가하고 포커스를 아래 셀로 이동합니다.
5 # Ctrl+Enter로 실행하면 현재 셀을 실행하고 포커스를 현재 셀에 머무르게 합니다.
6 # Alt+Enter로 실행하면 현재 셀을 실행하고 항상 아래에 새로운 셀을 만듭니다.

1 # 셀의 위쪽 또는 아래쪽 선 중간에 마우스를 올리면 '+ 코드'와 '+ 텍스트' 버튼이 보입니다.
2 # 새로운 코드 셀을 추가하려면 '+ 코드' 버튼을 클릭하세요.

+ 코드 + 텍스트

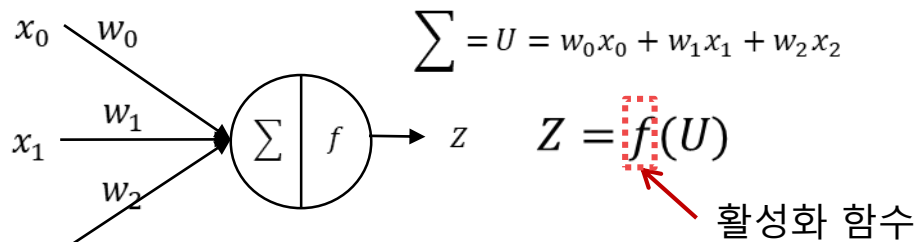
인공신경망 딥러닝 모델 구현

```
[ ] 1 from sklearn import datasets  
    2 iris = datasets.load_iris()  
    3 iris  
  
    {'data': array([[5.1, 3.5, 1.4, 0.2],  
                   [4.9, 3.1, 1.4, 0.2],  
                   [4.7, 3.2, 1.3, 0.2],  
                   [4.6, 3.1, 1.5, 0.2],  
                   [5. , 3.6, 1.4, 0.2],  
                   [5.4, 3.9, 1.7, 0.4],
```

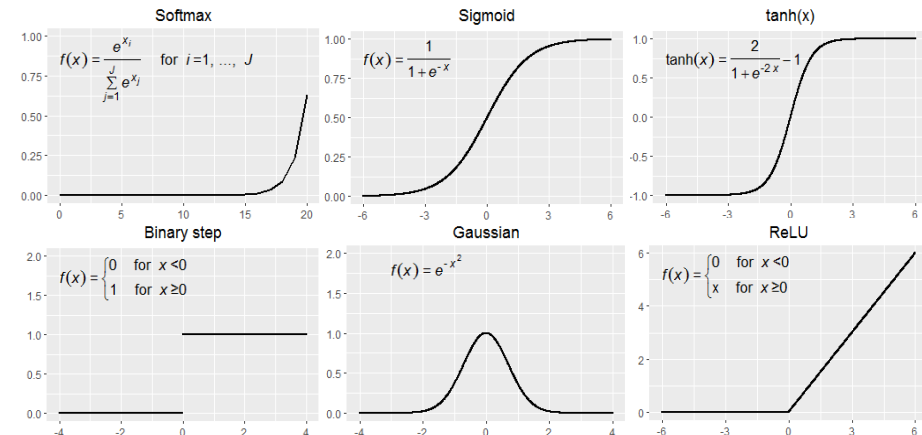
- Shift+Enter로 실행하면 아래에 셀이 없으면 새로운 셀을 추가하고 포커스를 아래 셀로 이동합니다.
- Ctrl+Enter로 실행하면 현재 셀을 실행하고 포커스를 현재 셀에 머무르게 합니다.
- Alt+Enter로 실행하면 현재 셀을 실행하고 항상 아래에 새로운 셀을 만듭니다.

- 셀의 위쪽 또는 아래쪽 선 중간에 마우스를 올리면 '+ 코드'와 '+ 텍스트' 버튼이 보입니다.
- 새로운 코드 셀을 추가하려면 '+ 코드' 버튼을 클릭하세요.

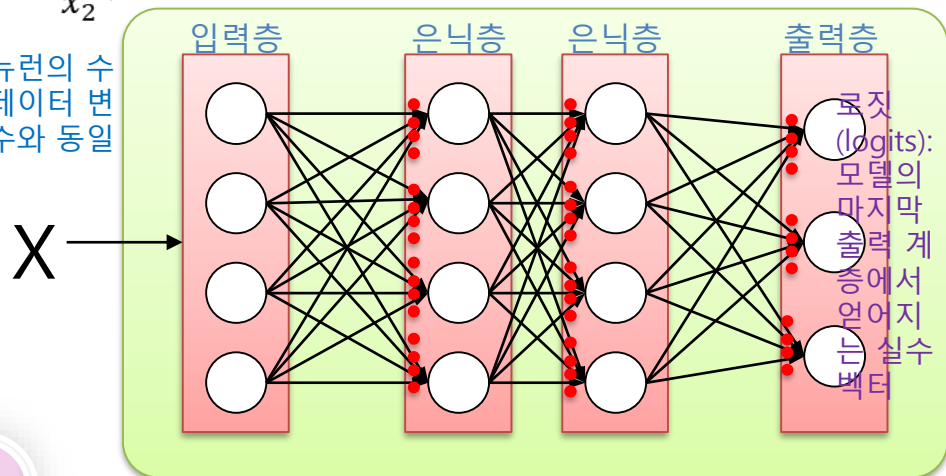
- #은 주석입니다.
- 코드를 작성하는 이 곳을 셀(Cell)이라고 부릅니다.
- 코드를 실행하려면 셀의 왼쪽에 있는 셀 실행 버튼을 클릭하거나 Shift+Enter 키를 누르세요.



소프트맥스 함수: 로짓을
입력으로 받아 각 클래
스에 대한 확률을 계산
하는 함수



입력층 뉴런의 수
는 입력데이터 변
수의 개수와 동일

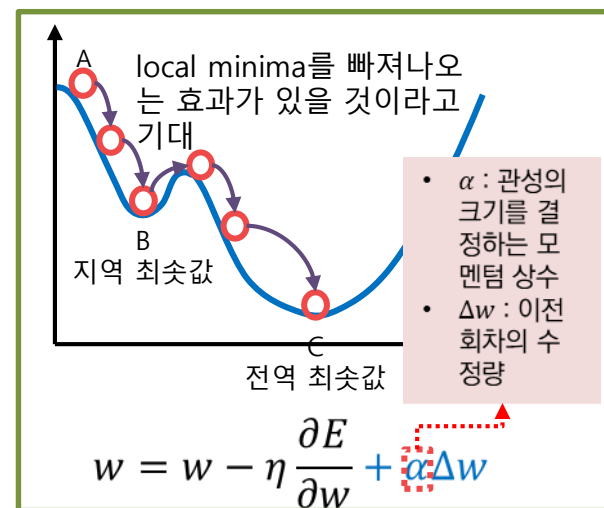
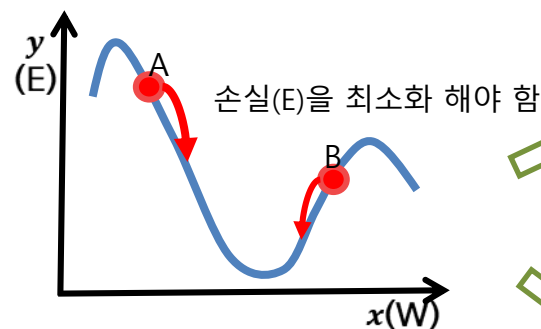


출력층 뉴런의 수는 분류
분석의 경우 분류 레이블
의 수이며, 회귀분석의
경우 1개

$\hat{y} \approx y$

차이(Loss, Error) { MAE, MSE, Crossentropy } 분류

손실함수



연구자가 고려해야 할 사항

- ▶ 은닉층의 수
- ▶ 은닉층별 뉴런의 수
- ▶ 활성화 함수
- ▶ 가중치 초기값
- ▶ 드롭아웃 비율
- ▶ 배치정규화
- ▶ 손실함수
- ▶ 옵티마이저, 학습률
- ▶ 학습 횟수(epochs)
- ▶ 배치 크기

가중치 업데이트

Gradient Descent

- weight
- 네트워크의 파라미터

$$w = w - \eta * \frac{\partial E}{\partial w}$$

- w에 대한 E의 편미분
- gradient
- 어떤 방향으로 학습할지

- Eta(learning rate)
- 한번에 얼마나 학습할지

- 편미분(偏微分, partial derivative)을 의미
- 기호는 델타의 변형
- 편-디(partial dee), 파셜디라고 부름

ChatGPT-3의 파라미터 개수는? 175B
ChatGPT-4의 파라미터 개수는? 1.7T

2장. 텐서플로우(케라스)를 이용한 딥러닝 구현

인공신경망 딥러닝 알고리즘 구현과 실시간 객체 탐지

1절. 텐서플로우 케라스

2장. 텐서플로우 케라스를 이용한 딥러닝 구현



케라스와 파이토치 비교

케라스와 파이토치 비교



Keras

- 높은 수준의 API를 제공함
- 코드가 직관적이고 사용하기 쉬움
- TensorFlow의 고수준 API로 통합되었음
- TensorFlow 생태계를 잘 활용할 수 있음



PyTorch

- 낮은 수준의 API를 제공
- 더 유연하고 커스터마이징하기 좋음
- 연구 커뮤니티에서 강력한 지지를 받고 있음
- 연구 결과를 빠르게 구현할 수 있는 라이브러리나 도구가 많음

기능	케라스	파이토치
모델 정의	Sequential 또는 Functional API 사용	nn.Module 클래스 상속
레이어 추가	model.add 또는 리스트 내 정의	__init__ 메서드 내에 레이어 정의
순전파	자동으로 처리	forward 메서드 내에서 직접 정의
모델 컴파일	model.compile 사용	없음, 직접 손으로 해야 함
모델 훈련	model.fit 사용	훈련 루프 직접 작성
손실 함수	loss 인자로 지정	nn.CrossEntropyLoss 등 손실 함수를 직접 정의
최적화 알고리즘	optimizer 인자로 지정	optim.Adam 등 옵티마이저를 직접 정의

텐서플로우 케라스

1절. 텐서플로우 케라스



텐서플로우 케라스

- ▶ 텐서플로우의 케라스 API는 딥러닝 모델을 구축하고 훈련하기 위한 고수준 API로, 다양한 신경망 구조를 쉽게 구축할 수 있도록 설계되었음
- ▶ 텐서플로우 2.x이후부터 공식적으로 케라스를 포함함

- **tf.keras.Sequential**: 순차 모델은 여러 층을 연이어 쌓아서 신경망을 구축하는 데 사용됨. 각 층은 이전 층의 출력을 입력으로 받음
- **tf.keras.Model**: 사용자 정의 모델을 만들 때 이 클래스를 상속하여 모델을 정의할 수 있습니다. 이를 사용하면 복잡한 모델 구조를 구현할 수 있음.
- **tf.keras.layers**: 이 모듈은 다양한 종류의 층을 포함하고 있으며, 이를 사용하여 모델의 구조를 정의함. 예를 들어, **Dense** 층은 완전 연결된 층을 생성하고, **Conv2D** 층은 합성곱 신경망을 생성함
- **tf.keras.losses**: 손실 함수를 정의하는 데 사용. 모델의 훈련 중에 사용되며, 훈련 중에 최소화하고자 하는 목표 함수를 정의함. 예를 들어, **categorical_crossentropy**는 다중 클래스 분류 문제의 크로스 엔트로피 손실 함수임
- **tf.keras.optimizers**: 최적화 알고리즘을 선택하고 모델을 훈련하기 위해 사용. 예를 들어, **Adam**, **SGD**, **RMSprop**와 같은 최적화 알고리즘을 제공함.
- **tf.keras.metrics**: 모델의 성능을 평가하기 위한 메트릭을 정의하는 데 사용. 예를 들어, 정확도(**accuracy**)나 정밀도(**precision**)를 계산할 수 있음
- **tf.keras.callbacks**: 모델 훈련 중에 호출되는 콜백 함수를 정의하는 데 사용. 예를 들어, 학습률을 동적으로 조정하거나 모델 저장을 자동화하는 데 유용함
- **tf.keras.utils**: 데이터 전처리 및 유틸리티 함수를 제공하는 모듈. 데이터 로드, 정규화, 원-핫 인코딩 등의 작업을 처리할 때 유용

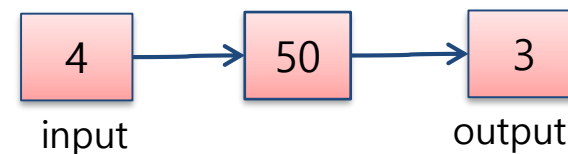
Sequential 클래스 이용

1절. 텐서플로우 케라스 > 1.1. 인공신경망 모델을 만드는 방법



예

```
1. from tensorflow.keras import Sequential
2. from tensorflow.keras.layers import Input, Dense
3.
4. model = Sequential()
5. model.add(Input(shape=4))
6. model.add(Dense(50, activation='relu'))
7. model.add(Dense(3, activation='softmax'))
8. model.summary()
```



Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	250
dense_1 (Dense)	(None, 3)	153

=====
Total params: 403 (1.57 KB)
Trainable params: 403 (1.57 KB)
Non-trainable params: 0 (0.00 Byte)



장점

- Sequential API로 모델을 설계하는 것은 직관적이고 편리함



단점

- 단순히 층을 쌓는 것만으로는 모든 인공신경망을 구현할 수 없음. 즉, 복잡한 인공 신경망을 구현할 수 없음.

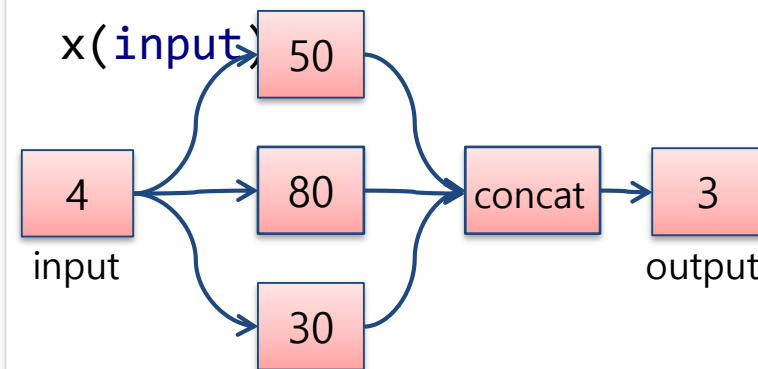
함수형 API 이용

1절. 텐서플로우 케라스 > 1.1. 인공신경망 모델을 만드는 방법

✓ 함수형 API를 이용하면 다중 입력 및 다중 출력 모델 생성 가능

1. `from tensorflow.keras import Model`
2. `from tensorflow.keras.layers import Input, Dense`
3. `from tensorflow.keras.layers import concatenate, Activation`
- 4.
5. `input = Input(shape=(4,))`
6. `dense1 = Dense(50, activation='relu')(input)`
7. `dense2 = Dense(80, activation='relu')(input)`
8. `dense3 = Dense(30, activation='relu')(input)`
9. `x = concatenate([dense1, dense2, dense3])`
10. `output = Dense(3, activation='softmax')(x)`
11. `model = Model(inputs=input, outputs=output)`
12. `model.summary()`

`x = Dense(64)(input)`
위의 코드는 아래의 코드와
같음
`x = Dense(64)`



Residual block 구현

